

SQL PLACEMENT PREPARATION NOTES

Modules 26–28 • Primary Key • Foreign Key • Constraints

Purpose: Dedicated beginner-friendly notes for understanding keys and constraints before moving deeper into SQL joins and placement problems.

26. PRIMARY KEY

1. Definition

A **Primary Key** is a column, or combination of columns, that uniquely identifies each row in a table.

Example:

```
CREATE TABLE Employee (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  salary INT  
);
```

Here, **emp_id** uniquely identifies every employee.

2. Rules / Properties

Rule	Meaning
Unique	Two rows cannot have the same primary-key value.
NOT NULL	A primary-key value cannot be NULL.
One primary key constraint	A table has one PRIMARY KEY constraint, although that constraint can contain multiple columns.
Stable identifier	It should identify a row reliably and should generally not change unnecessarily.
Can be referenced	Other tables can use it as a foreign-key reference.

3. Simple Example

emp_id	name	department
101	Anu	IT
102	Ravi	HR
103	Kiran	IT

Here **emp_id** is the primary key. 101, 102 and 103 must be different and cannot be NULL.

4. Why Do We Need a Primary Key?

It provides a reliable way to identify a particular row and helps establish relationships with other tables.

For example, if two employees have the same name, emp_id still distinguishes them.

5. Composite Primary Key

A **composite primary key** uses two or more columns together to uniquely identify a row.

Example:

```
CREATE TABLE Enrollment (  
  student_id INT,  
  course_id INT,  
  PRIMARY KEY (student_id, course_id)  
);
```

Here, **student_id alone** can repeat and **course_id alone** can repeat. But the combination (**student_id, course_id**) must be unique.

Example concept:

student_id	course_id	Valid?
1	101	Yes
1	102	Yes
2	101	Yes
1	101	No — duplicate combination

6. Primary Key vs Unique Key

Primary Key	UNIQUE
Uniquely identifies the row.	Ensures values are unique for the constrained column(s).
PRIMARY KEY implies NOT NULL.	NULL handling can differ by DBMS; UNIQUE is not the same as PRIMARY KEY.
One PRIMARY KEY constraint per table.	A table can have multiple UNIQUE constraints.
Commonly used as the main row identifier.	Useful for alternate/candidate identifiers such as email.

■ Placement memory

PRIMARY KEY = UNIQUE + NOT NULL + main row identifier.

Composite key = multiple columns together identify one row.

27. FOREIGN KEY

1. Definition

A **Foreign Key** is a column or set of columns in one table that references a candidate key—commonly the primary key—of another table.

Its main purpose is to maintain a valid relationship between related tables.

2. Why Is a Foreign Key Used?

It helps enforce **referential integrity**: a child row should not normally contain a foreign-key value that does not correspond to a valid referenced parent row, subject to the database's constraints and actions.

3. Parent Table and Child Table

Parent table: The table containing the referenced key.

Child table: The table containing the foreign key.

Example:

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

Department is the parent table. **Employee** is the child table. Employee.dept_id references Department.dept_id.

4. Relationship Diagram

Department (Parent)	Employee (Child)
dept_id ← referenced key	dept_id ← foreign key
IT = 10	Anu = 10
HR = 20	Ravi = 20

The value 10 in Employee.dept_id must correspond to a valid Department.dept_id when the foreign-key constraint is enforced and no special NULL/action behavior applies.

5. Foreign Key vs Primary Key

Primary Key	Foreign Key
Identifies rows in its own table.	Creates/references a relationship to another table.
Must be unique.	Does not have to be unique.
Cannot be NULL.	Can be NULL unless NOT NULL is also specified.
One PRIMARY KEY constraint per table.	A table can have multiple foreign-key constraints.
Referenced by foreign keys.	References a key in another table.

6. Referential Integrity

Referential integrity means the relationship between parent and child tables remains valid according to the foreign-key rules.

Example: If Department 10 does not exist, inserting an Employee with dept_id = 10 will normally fail when the foreign-key constraint is active and no valid special handling applies.

7. ON DELETE

Defines what happens to child rows when the referenced parent row is deleted. Common actions include:

Action	Meaning
CASCADE	Delete related child rows automatically.
SET NULL	Set the child foreign-key value to NULL, if allowed.
RESTRICT / NO ACTION	Prevent the parent deletion when related child rows exist, depending on DBMS behavior.

Example:

```
FOREIGN KEY (dept_id) REFERENCES Department(dept_id) ON DELETE CASCADE
```

If a department is deleted, its related employee rows are also deleted under CASCADE. Use such actions carefully.

8. ON UPDATE

Defines what happens when a referenced key value in the parent table is updated. A common action is **CASCADE**, which updates matching foreign-key values in child rows.

Example:

```
FOREIGN KEY (dept_id) REFERENCES Department(dept_id) ON UPDATE CASCADE
```

Exact supported actions and syntax can vary between database systems.

■ Placement memory

Parent has referenced key → Child has foreign key.

FK does not have to be unique.

Referential integrity = valid parent-child relationship.

28. CONSTRAINTS

1. Definition

Constraints are rules applied to table columns or tables to control what data can be stored and to maintain data integrity.

Think of a constraint as a **rule the database checks** when data is inserted or modified.

2. PRIMARY KEY Constraint

Uniquely identifies each row and does not allow NULL values.

```
CREATE TABLE Student (student_id INT PRIMARY KEY, name VARCHAR(50));
```

3. FOREIGN KEY Constraint

Connects a child table to a referenced key in a parent table.

```
FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
```

4. NOT NULL Constraint

Prevents a column from containing NULL values.

```
CREATE TABLE Employee (emp_id INT, name VARCHAR(50) NOT NULL);
```

This means an Employee row must have a value for name.

5. UNIQUE Constraint

Prevents duplicate values for the constrained column or combination of columns. NULL behavior can differ by DBMS.

```
CREATE TABLE UserAccount (user_id INT PRIMARY KEY, email VARCHAR(100) UNIQUE);
```

6. DEFAULT Constraint

Provides a default value when an INSERT does not supply a value for that column.

```
CREATE TABLE Employee (  
emp_id INT PRIMARY KEY,  
status VARCHAR(20) DEFAULT 'Active'  
);
```

If status is omitted in an applicable INSERT, the database can use 'Active'.

7. CHECK Constraint

Requires a value to satisfy a specified condition.

```
CREATE TABLE Employee (  
emp_id INT PRIMARY KEY,  
salary INT CHECK (salary >= 0)  
);
```

The condition prevents values that violate the check, subject to DBMS rules.

8. AUTO_INCREMENT / Identity Concept

Used to generate sequential numeric identifiers automatically. The exact keyword differs by DBMS.

MySQL-style example:

```
CREATE TABLE Employee (emp_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
```

Other systems use concepts such as **IDENTITY** or sequences. Do not assume AUTO_INCREMENT is universal SQL syntax.

9. Complete Example Using Several Constraints

```
CREATE TABLE Employee (  
emp_id INT PRIMARY KEY,
```

```

name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE,
salary INT CHECK (salary >= 0),
status VARCHAR(20) DEFAULT 'Active',
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

```

10. Constraint Comparison Table

Constraint	Main job	Simple memory
PRIMARY KEY	Unique row identifier; not NULL.	Main identity
FOREIGN KEY	Maintains relationship to referenced table.	Connect tables
NOT NULL	Value cannot be NULL.	Must provide value
UNIQUE	Prevents duplicate constrained values.	No duplicates
DEFAULT	Supplies a value when one is omitted.	Automatic default
CHECK	Requires a condition to be satisfied.	Must pass condition
AUTO_INCREMENT / IDENTITY	Generates identifier values automatically.	Automatic ID

11. Common Placement Traps

Trap 1: Primary key is not just UNIQUE; it also cannot be NULL.

Trap 2: Foreign key values do not have to be unique. Many employees can belong to the same department.

Trap 3: A table can have only one PRIMARY KEY constraint, but it can contain multiple columns and a table can have multiple UNIQUE/FK constraints.

Trap 4: UNIQUE and PRIMARY KEY are not interchangeable. NULL behavior for UNIQUE can vary by DBMS.

Trap 5: FOREIGN KEY usually references a primary key or another candidate/unique key depending on DBMS rules; it does not mean the child value itself must be unique.

Trap 6: DELETE and TRUNCATE are different SQL operations; a foreign key can affect whether DELETE operations are allowed or cascaded.

Trap 7: AUTO_INCREMENT is DBMS-specific terminology. Other databases use IDENTITY or sequences.

12. Quick Interview Questions

Question	Answer to remember
Can a primary key contain NULL?	No.
Can a foreign key contain duplicate values?	Yes.
Can a foreign key be NULL?	Yes, unless restricted by NOT NULL or other rules.
Can a table have multiple primary keys?	No—one PRIMARY KEY constraint; it may be composite.
Can a table have multiple foreign keys?	Yes.
Can a composite primary key have repeated individual column values?	Yes—the combination must be unique.
What maintains parent-child relationship?	Foreign key + referential integrity.
What automatically supplies a value?	DEFAULT.
What checks a condition?	CHECK.
What prevents NULL?	NOT NULL.

13. One-Minute Revision

PRIMARY KEY → uniquely identifies a row + NOT NULL.

FOREIGN KEY → connects child to parent.

NOT NULL → value required.

UNIQUE → duplicates not allowed.

DEFAULT → automatic value when omitted.

CHECK → condition must be satisfied.

AUTO_INCREMENT / IDENTITY → automatic numeric identifier generation.

Placement formula:

PRIMARY KEY = **Identity** | FOREIGN KEY = **Relationship** | NOT NULL = **Required** | UNIQUE = **No duplicate** |

DEFAULT = **Fallback value** | CHECK = **Condition** | AUTO_INCREMENT = **Automatic ID**

These three modules should now be added to your main SQL placement sequence before continuing with advanced JOIN and coding practice.